PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Intelligent Digital Library and Book Recommendation System

Team Name: *Team # (Team ID):*	*[Write your team name here]* *DSCPP-II-2026-T367*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Goyal, Vanshika – 2510376201* *Vanshikagoyal701@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Singhal, Pragati – 2510010644* *pragatisinghal05@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Bugana, Prateek – 2510375052* *buganaprateek0274@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

Our project is a library program that runs in the console, made for a college. A student can search for a book, issue it, return it, and check which books he or she has read before. The program also suggests new books. It looks at the genre and the author of the books a student already read and picks similar ones.
The admin can add books, delete books, see which books are out on loan, and make new student accounts.
The main logic is written in C: a linked list, a hash table, a binary search tree, sorting and file handling. The login, the Student and Admin roles, the menus and the book suggestions are written in C++ with classes, inheritance and polymorphism. Everything is saved in plain text files, so we did not need a database.
With this project we wanted to show how data structures and object oriented programming fit together in one small but working application.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

The program is split into two parts. We compile them separately and join them at the end.
Part 1 is in C (library.c). All books sit in a linked list. A hash table (book ID modulo 101) finds a book by its ID quickly, and a binary search tree finds a book by its title. We can sort the books by title, author or popularity using bubble sort. Books, users and issue history are stored in three text files, and the | sign separates the fields in each line.
Part 2 is in C++ (main.cpp). There is an abstract class `User` with two child classes, `Student` and `Admin`. The same call, `u->menu()`, opens the right menu for whoever logged in. A `Recommender` class gives 2 points to a book with the same genre as one the user read, and 3 points for the same author. The top 5 books are shown.
C++ uses the C functions through `library.h` (with `extern "C"`). We use GCC for C and G++ for C++, a Makefile, and Code::Blocks. We did not use any outside library, only the standard C and C++ ones.

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Linked list for the books, adding and deleting a book, saving and loading the books file	Vanshika Goyal
Issue and return of books, reading history file, admin report of issued books	Vanshika Goyal
Search by book ID (hash table) and search by title (binary search tree, full or partial name)	Pragati Singhal
Sorting by title, author and popularity; a test script with 17 automatic tests	Pragati Singhal
Classes User, Student and Admin; login with roles; the two menus	Prateek Bugana
Book suggestions from genre, author and reading history	Prateek Bugana
Report, viva questions, run guide, debugging guide and the GitHub repository	All members

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

- **Joining C and C++.** In the beginning the linker showed "undefined reference". C++ changes the names of functions, so it could not find our C functions. Adding `extern "C"` in `library.h` fixed it.
- **Wrong lines in the data files.** If a line had a missing | sign, or came from Windows with a different line ending, loading failed. We wrote one helper, `splitLine`, which cuts a line, skips the bad ones and removes the line ending. We also wrote tests for this.
- **Memory problems.** A very long text could overflow an array, and a book that does not exist could crash the program with NULL. Now we copy text only up to the array size and check every search result. valgrind and AddressSanitizer both show 0 errors.
- **The first code was too hard to explain.** It had many rule classes and insertion sort. We changed it to one simple recommender, bubble sort and fixed size arrays, so each of us can explain it in the viva.
- **Things we know are weak.** The tree is not balanced and the passwords are stored as plain text. We will write them as future work.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
Run the project once on Windows in Code::Blocks and fix any problem	All members
Make the final presentation slides	Vanshika Goyal
Practice the live demo (search, sort, issue, return, suggestions, admin)	Pragati Singhal and Prateek Bugana
Viva practice: each of us explains our own part of the code	All members

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

At the end we will have a working console program with these parts:

- Source code: `library.h`, `library.c` (C) and `main.cpp` (C++), with a comment on every line.
- Sample data files for books, users and issue history (25 books and 3 users).
- A Makefile and a test script (`make test`) that runs 17 checks.
- The project report, viva questions and answers, a “how to run” guide and a debugging guide.
- The GitHub repository and the final presentation.

Progress Overview (2 pts)

(Summarize how much of the project is done, what’s behind schedule, what’s ahead of schedule. Max 200 words.)

Every feature we promised in the Phase-I presentation is done: linked list, file handling, hashing, binary search tree, sorting, classes, inheritance, polymorphism, Student and Admin roles, and the recommendation system. The code builds with no warnings and all 17 tests pass.
We are not behind on anything important. We even did a few extra things that were not planned: a debugging guide, a one-command test (`make test`) and a memory check with valgrind. Three things are left: the final presentation, practice of the live demo, and one more run on Windows.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository: `https://github.com/pragatisinghal05-cyber/smart-library-pbl` (public)
Branch: `main`

Important commits:

- `[commit id]` – simplified the code (bubble sort, simple recommender, `splitLine`, fixed arrays)
- `[commit id]` – merged the code into two files (`library.c`, `main.cpp`) and added `make test`
- `[commit id]` – comment added on every line of code
- `[commit id]` – longer explanations, examples and pictures in the comments

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
Login (wrong and correct password)	Pass	Wrong password is not accepted
Search by title (tree), full and partial name	Pass	"harry" gives 2 books in A to Z order
Search by ID (hash table) and sorting (title, author, popularity)	Pass	ID 999 gives a clear message; first book is correct in all 3 sort modes
Issue and return a book	Pass	Second issue of the same book is refused
Recommendation	Pass	Read books are not suggested; new student gets a message
Admin: add, delete, issued list, new account	Pass	Duplicate ID and deleting an issued book are refused
Wrong input, very long word, Windows line endings, missing file	Pass	Program keeps running, no crash
Memory check (valgrind, AddressSanitizer) and compiler warnings (-Wall -Wextra)	Pass	0 errors and 0 warnings

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

- Source code (C and C++) – **Completed**
- Data files, Makefile and test script (17 tests) – **Completed**
- Project report, viva Q and A, run guide, debugging guide – **Completed**
- GitHub repository – **Completed**
- Check on Windows with Code::Blocks – **Pending**
- Final presentation and live demo – **In progress**